

מכללת אורט סינגאלובסקי

מחלקת תוכנה — מסלול טכנאי תוכנה

ספר פרויקט

BloomStore

חנות פרחים מקוונת — Full-Stack E-Commerce

מוגשת על ידי

אביב לייסטן · ת"ז 206917353

050-5806570

בהנחיית

מור ברגיג

052-8612379

דרך הטייסים 28, תל אביב

22 בספטמבר 2026

תוכן עניינים

פרק 1 — מבוא ותיאור הפרויקט

1.1 מטרת המערכת

1.2 חלוקת תפקידים ומבנה הספר

פרק 2 — ארכיטקטורה ומודל הנתונים

2.1 טכנולוגיות

2.2 מודל הנתונים

פרק 3 — דרישות פונקציונליות

פרק 4 — צד הלקוח: יסודות הממשק

4.1 ממשק החנות — מבט מקדים

4.2 הקובץ App.tsx וספקי ה-state

4.3 CatalogPage.tsx — דף הבית

4.4 ProductCard.tsx — כרטיסיית מוצר

4.5 עיצוב הפרויקט

פרק 5 — צד השרת: מודול ההזדהות

5.1 שלוש השכבות

5.2 התחברות ונעילת חשבון · POST `api/auth/login/`

5.3 אימות GET · `api/auth/me/` Token

5.4 שאר מנגנוני המודול

5.5 החיבור לצד הלקוח

פרק 6 — צד השרת: מודול הניהול

6.1 שלוש השכבות

6.2 אישור וביטול הזמנה

6.3 שינוי סטטוס הזמנה · PATCH `id/status:/api/orders/`

6.4 שאר מנגנוני המודול

6.5 החיבור לצד הלקוח

פרק 7 — עגלה, הזמנה ומחזור חיים

7.1 עגלת קניות

7.2 יצירת הזמנה

7.3 מחזור חיים של הזמנה

פרק 8 — נקודות הקצה

8.1 נקודות הקצה

פרק 9 — תהליך הפיתוח, בדיקות והפצה

9.1 תהליך הפיתוח

9.2 מערך הבדיקות

9.3 תהליך ההפצה

פרק 10 — סיכום ומסקנות

10.1 הישגים טכניים

10.2 אתגרים ומה שנלמד

10.3 פיתוחים עתידיים

10.4 תודות

מקורות

הצהרת הסטודנטית

אני מצהירה כי העבודה המוגשת להלן נעשתה על ידי באופן עצמאי, על פי ידיעתי האישית, תוך שימוש בקוד, במאמרים ובמקורות אחרים על פי כללי האתיקה האקדמית ובציון מקורות מתאימים. הספר מתאר את BloomStore כפי שהיא בנויה בפרויקט הזה.

פרק 1 — מבוא ותיאור הפרויקט

הפרויקט BloomStore הוא חנות פרחים מקוונת מלאה (Full-Stack E-Commerce), והוא נבנה כפרויקט גמר במסלול טכנאי תוכנה. המערכת מכסה את מסלול הרכישה כולו: עיון בקטלוג ציבורי, הרשמה והתחברות, עגלת קניות, בחירת כתובת משלוח, אישור ההזמנה, ומעקב אחרי סטטוס המשלוח. מנהל החנות מנהל מלאי, מקדם הזמנות וצופה בלוח סטטיסטיקות.

המערכת חיה במונורפו אחד (Nx): אפליקציית React, שרת NestJS, וספריית חוזים משותפת ב-TypeScript. שני הצדדים רצים בנפרד ומתקשרים דרך חוזה API מתועד ב-Swagger.

1.1 מטרות המערכת

- מערכת מסחר אלקטרוני פונקציונלית מקצה לקצה, מרישום משתמש ועד אישור הזמנה.
- ארכיטקטורת שרת-לקוח מופרדת בתוך מונורפו, עם חוזה טיפוסים אחד ב-`libs/shared-types`.
- אכיפת כל פעולה רגישה בצד השרת, ולא רק הסתרה בצד הלקוח.
- לוח ניהול למוצרים, להזמנות ולתמונות מצב סטטיסטי.
- מלאי עקבי: נעילת מחיר בשורת ההזמנה, והחזרת מלאי בטרנזקציה כשמבטלים לפני משלוח.
- עגלה שנשמרת במסד, כך שהיא נשארת גם אחרי סגירת הדפדפן.
- עיצוב אחיד של הממשק באמצעות ספריית MUI.

1.2 חלוקת תפקידים ומבנה הספר

ההגשה היא של סטודנטית יחידה.

רכיב	מפתחת	תחום אחריות
<code>apps/frontend</code>	אביב לייסטן	ממשק משתמש, ניהול state ועיצוב ב-MUI
<code>apps/backend</code>	אביב לייסטן	שרת, מסד נתונים, אימות, הזמנות, אבטחה
<code>libs/shared-types</code>	אביב לייסטן	חוזה API, סכמות Zod, מכונת מצבים
<code>docs/project-book.md</code>	אביב לייסטן	ספר הפרויקט והתייעוד

הספר סוקר את המערכת כולה, ומעמיק בשלושה אזורים שנבחרו להצגה מפורטת: יסודות צד הלקוח (פרק 4), מודול ההזדהות בצד השרת (פרק 5) ומודול הניהול בצד השרת (פרק 6). שאר הפרקים מציגים את היקף המערכת, את מודל הנתונים, את תהליך ההפצה ואת מערך הבדיקות.

פרק 2 — ארכיטקטורה ומודל הנתונים

המערכת BloomStore בנויה על ארכיטקטורת שלוש שכבות: ממשק משתמש, לוגיקה עסקית ומסד נתונים. אותו עיקרון חוזר בתוך השרת. רכיב ה-Controller מחלץ פרמטרים ומחזיר מעטפת ApiResponse. הוא אינו ניגש למסד הנתונים. אימות Zod נעשה בקונטרולר בהזדהות, ובשירות בשאר המודולים. רכיב ה-Service מחזיק את כללי המלאי, ההזמנה והאבטחה. רכיב ה-Model של Mongoose הוא שכבת ההתמדה.

2.1 טכנולוגיות

שם מה	טכנולוגיה	שכבה
פרויקט אחד לשלוש חבילות, בנייה ובדיקות דרך <code>npx nx</code>	TypeScript · Nx במצב strict · Node 20+	מונורפו
רכיבים לשימוש חוזר וטיפוסים משותפים עם השרת	React 19 ו-TypeScript	צד לקוח
שרת פיתוח ומעבר בין דפים בלי רענון	Vite ו-React Router 7	צד לקוח
צירוף אוטומטי של ה-JWT לכל בקשה	Axios	צד לקוח
ספריית רכיבי הממשק	MUI 7	צד לקוח
צבע, גופן ומבנה מסך אחידים דרך ספריית MUI	עיצוב החנות	צד לקוח
תרשים מכירות בלוח הניהול	Recharts	צד לקוח
מודולים והרשאות גישה	NestJS 11 על Express	צד שרת
מסמכי הזמנה, סכמה ואינדקסים	MongoDB ו-Mongoose	צד שרת
עותק קצר של העגלה. MongoDB נשאר מקור האמת	מטמון בזיכרון התהליך	צד שרת
בדיקת קלט, כותרות אבטחה ומקורות מורשים	Zod · Helmet · CORS	צד שרת
גיבוב סיסמאות והנפקת טוקן גישה	bcryptjs · @nestjs/jwt	צד שרת
תיעוד חי של ה-API ב-/api/swagger	Swagger	צד שרת
אשכול עם העתקים, נדרש לטרנזקציית הביטול	MongoDB Atlas	נתונים
בדיקות אוטומטיות, אתר החנות, וה-API בקונטיינר	GitHub Actions · GitHub Pages · Render	הפצה

טבלה 2 — הטכנולוגיות שנבחרו והתפקיד של כל אחת

2.2 מודל הנתונים

הנתונים מאורגנים באחת-עשרה קולקציות: שש קולקציות ליבה עסקית וחמש קולקציות תמיכה ותשתית.

קולקציה	תפקיד	שדות מרכזיים
users	חשבון, הרשאות ואבטחה	,passwordHash, email ,role, fullName ,failedLoginAttempts lockUntil
products	קטלוג	,description, name ,stock, price, category ,isActive, imageUrl stockNotifyEmails
carts	עגלה אחת לכל משתמש (1:1)	,userId items [{productId, quantity}]
orders	הזמנה עם Snapshot של הרכישה	,userId, orderNumber ,total, items[], status addressId
payments	סכום שנרשם עם ההזמנה	,userId, orderId status, amount
addresses	פנקס כתובות משלוח	,fullName, userId ,street, city, phone ,houseNumber isDefault
sequences	מונה אטומי למספרי הזמנה	value, name (ה-seed מתחיל את המונה ב-1000; מספר ההזמנה הוא n-BLM)
auditlogs	יומן ביקורת	,action, actorId ,entityId, entity metadata
webhookevents	אירוע חיצוני שכבר נקלט	,type, eventId status, payload
failedwebhooks	אירוע שנכשל בקליטה	,reason, type, eventId payload
idempotencykeys	מניעת הזמנה כפולה	,orderId, userId, key תפוגה אחרי 24 שעות

טבלה 3 — אחת-עשרה הקולקציות במסד הנתונים

יחסים: משתמש אחד לרבות כתובות והזמנות, משתמש אחד לעגלה אחת, הזמנה אחת למסמך סכום אחד. שורות ההזמנה שומרות עותק של שם המוצר ושל המחיר. הן אינן תלויות במחיר החי בקטלוג אחרי יצירת ההזמנה.

אינדקס הקטלוג הוא על `isActive`, `stock` ו-`category`. מוצר עם `isActive: false` נעלם מהקטלוג הציבורי. מוצר עם `stock === 0` נשאר גלוי, עם הרשמה להתראת מלאי, והעגלה עדיין דוחה אותו.

פרק 3 — דרישות פונקציונליות

המערכת כוללת אחד-עשר מסכים בשלושה אזורים: אזור ציבורי, אזור למשתמש מחובר, ולוח ניהול.

מסך	תיאור	פונקציות מרכזיות
דף הבית	קטלוג המוצרים הפעילים	שליפה מהשרת, סינון קטגוריה, גריד, מוצרים שאזלו נשארים עם התגית Out of stock וטופס Email me
דף מוצר	פרטי פריט בודד	תמונה, מחיר, מלאי, הוספה לעגלה או טופס התראת מלאי
יצירת קשר	/contact	פרטי החנות וטופס פנייה
התחברות	כניסת משתמש קיים	ולידציית שדות, שמירת Token, הודעת נעילה ב-423
הרשמה	יצירת חשבון לקוח	ולידציה, יצירת משתמש, כניסה אוטומטית
עגלת קניות	פריטים לפני ההזמנה	שינוי כמות, הסרת פריט, מעבר לקופה
קופה	כתובת ויצירת הזמנה	בחירה או יצירה של כתובת, מפתח idempotency, סטטוס pending_payment
ההזמנות שלי	מעקב הלקוח	מספר הזמנה, מחיר נעול, סטטוס, ביטול לפני משלוח
ניהול — מוצרים	קטלוג כולל לא-פעילים	הוספה, סימון אזל, החזרה למלאי, מחיקה רכה
ניהול — הזמנות	מעקב מנהל	סינון לפי סטטוס וקידום לפי מכונת המצבים
ניהול — סטטיסטיקות	תמונת מצב	הכנסות, הזמנות פתוחות, מלאי נמוך, צמיחת משתמשים ותרשים שבועי

טבלה 4 — אחד-עשר מסכי המערכת והפונקציות שבכל אחד

דרישות לא פונקציונליות: TypeScript strict בכל הפרויקטים, מעטפת JSON אחידה { success, data } או { success: false, error }, ותיעוד חי של הספר ושל ה-API.

פרק 4 — צד הלקוח: יסודות הממשק

פרק זה נפתח במבט על הממשק שהלקוח רואה, ואחריו מציג את מעטפת האפליקציה, את דף הקטלוג ואת כרטיסיית המוצר. עיצוב הפרויקט סוגר את הפרק.

4.1 ממשק החנות — מבט מקדים

דף הבית הוא נקודת הכניסה. המסגרת קבועה, והרשת שמתחתיה משתנה לפי מלאי.

אזור	מה מוצג	כלל
מסגרת	Header, תוכן, Footer	ניהול רק בתפריט החשבון, ורק ל-admin
ראש העמוד	CatalogHero	קריאה לפעולה Shop Flowers
סינון	CategoryPills	כפתורי קטגוריה מעל הרשת
כרטיס	תמונה, שם, מחיר, פעולה	המחיר בשקלים
אזל מהמלאי	תגית מלאי וטופס דוא"ל	הפריט נשאר על המדף
לא פעיל	אינו מוצג	השרת לא שולח את המוצר

טבלה 5 — מבנה הממשק בדף הבית

4.2 הקובץ App.tsx וספקי ה-state

הקובץ main.tsx מרכיב את העץ: ThemeProvider ו-CssBaseline לפי ערכת הנושא, BrowserRouter עם בסיס הנתוב של GitHub Pages, ואז AuthProvider ו-CartProvider. שני המצבים הגדולים — המשתמש המחובר והעגלה — יושבים ב-Context ולא בתוך App.tsx. כך כל דף קורא ל-useAuth או ל-useCart בלי להעביר props לאורך כל העץ.

הקובץ App.tsx אחראי על ארבעה דברים: גלילה לראש בכל ניווט, הצגת Header ו-Footer, הגדרת הנתובים, ושערי גישה. Private מפנה ל-/login כשאין משתמש. AdminOnly מפנה לדף הבית כשהתפקיד אינו admin. השער בצד הלקוח הוא חוויית משתמש. האכיפה נשארת בשרת.

הספק AuthProvider בודק בעליית האפליקציה אם יש bloomstore.token ב-localStorage. אם יש, הוא קורא ל-GET /api/auth/me. תשובה תקינה שומרת את המשתמש. כשל מוחק את הטוקן. התחברות והרשמה שומרות את ה-Token שחזר מהשרת. התנתקות קוראת ל-POST /api/auth/logout ומוחקת את המפתח המקומי.

הספק `AuthProvider` שולף את העגלה מ-`GET /api/cart` בכל פעם שיש משתמש מחובר. בלי משתמש העגלה המקומית מתרוקנת. הוספה, עדכון והסרה מעדכנים את ה-`state` לפי תשובת השרת.

לקוח ה-`Axios` ב-`services/api.ts` מצמיד `Authorization: Bearer` לכל בקשה שיש לה טוקן, ופורש את המעטפת `{ success, data }`.

4.3 CatalogPage.tsx — דף הבית

הדף `CatalogPage` אחראי על שליפת המוצרים, על שמירתם ב-`state`, על סינון לפי קטגוריה מתוך ה-`query string`, ועל הצגה באמצעות `ProductCard`. בזמן טעינה מוצגים שישה שלדי כרטיס (`ProductCardSkeleton`). רשימה ריקה מציגה `EmptyState`. הקטגוריה נקראת מ-`?category=`. ערך שאינו בקבוצת הקטגוריות החוקית מתעלמים ממנו. הבקשה היא `GET /api/products` או `GET /api/products?category=`. השרת כבר מסתיר מוצרים לא פעילים וממין פריטים במלאי לפני פריטים שאזלו. הדף אינו מסנן לבד מוצרים עם מלאי אפס, כדי שהלקוח יוכל להירשם להתראה.

apps/backend/src/products/products.service.ts

```
async listPublic(category?: string): Promise<ProductDto[]> {
  const filter: Record<string, unknown> = { isActive: true };
  if (category) {
    filter.category = category;
  }
  const rows = await this.products.find(filter).sort({ name: 1 });
  return rows
    .sort((a, b) => {
      const aOut = a.stock === 0 ? 1 : 0;
      const bOut = b.stock === 0 ? 1 : 0;
      if (aOut !== bOut) {
        return aOut - bOut;
      }
      return a.name.localeCompare(b.name);
    })
    .map((p) => this.toDto(p));
}
```

איור 1 — סינון הקטלוג ב-`listPublic`

הוספה לעגלה בודקת קודם אם יש משתמש. אורח מועבר ל-`/login` עם כתובת החזרה. משתמש מחובר קורא ל-`add` של ה-`Context`.

4.4 ProductCard.tsx — כרטיסיית מוצר

הכרטיס `ProductCard` מציג מוצר אחד: תמונה, שם, מחיר, ומצב מלאי. הוא מקבל את המוצר ואת `onAddToCart`. לחיצה על התמונה או על השם מובילה ל-`/products/:id`.

כשהמלאי אפס, במקום כפתור הוספה מופיע טופס התראת דוא"ל. בין יחידה אחת לשלוש מופיעה תגית של מלאי נמוך. בזמן ההוספה הכפתור ממתיך, ואחרי הצלחה יש אישור קצר. שגיאה מהשרת, למשל כמות גדולה מהמלאי, נשארת על הכרטיס.

apps/frontend/src/components/ProductCard.tsx

```
{outOfStock ? (
  <StockNotifyForm productId={product.id} defaultEmail={notifyEmail} compact />
) : (
  <Button size="small" variant="contained" disabled={adding} onClick={() => void handleAdd()}>
    {adding ? 'Adding...' : added ? 'Added' : 'Add to cart'}
  </Button>
)}
```

איור 2 — הכרטיס ב-ProductCard כשהמלאי אזל

בריוחף עם עכבר הכרטיס עולה מעט ומקבל צל רך. במסך מגע אין את האפקט הזה.

4.5 עיצוב הפרויקט

העיצוב חשוב מאוד, כי זה הדבר הראשון שרואים כשנכנסים לחנות. האתר מרגיש נקי ומסודר, ומגוון צבעי הפרחים יכול להופיע בלי שהעין תתעייף. הרקע קרם, הכרטיסים בהירים ורכים, הכפתור הראשי בורד מאובק, וההדגשות בירוק מרווה. הטקסט נשאר כהה וקריא, כדי שהצבע יישאר בתמונות ולא יתפזר על כל העמוד. הכותרות ב-Fraunces, גופן רך עם קצת אופי, והטקסט השוטף ב-DM Sans.

המסכים בנויים מרכיבי MUI, וכל עמוד עטוף ב-PageShell. הכפתורים לא כתובים באותיות גדולות, הפינות מעוגלות, והצל על הכרטיס עדין. לא נוסף צבע חדש בכל מסך. כל הערכים יושבים במקום אחד, ב-src/theme/tokens.ts, ומשם נצבעים רכיבי הספרייה.

פרק 5 — צד השרת: מודול ההזדהות

פרק זה מציג את מודול ההזדהות: תחילה את שלוש השכבות, ואחר כך שתי נקודות קצה במלואן.

5.1 שלוש השכבות

ב-NestJS אין קובץ ראوتر נפרד. הנתבים וה-Guards יושבים על הקונטרולר. השירות אינו מכיר את אובייקט ה-HTTP.

שכבה	קובץ	אחריות
Controller	auth/ auth.controller.ts	ארבע נקודות קצה: register, me, login, logout
Service	auth/auth.service.ts	הרשמה, התחברות, נעילה, הנפקת JWT
Model	models/ user.schema.ts	סכמת המשתמש
Guard	auth/jwt.strategy.ts	פענוח ה-Token מתוך כותרת Authorization
Rate limit	main.ts	עד 20 ניסיונות לדקה מאותה כתובת IP על login, register והתראת מלאי

טבלה 6 — חלוקת האחריות במודול ההזדהות

נתיבי register ו-login ציבוריים. נתיבי logout ו-me דורשים AuthGuard('jwt')

5.2 התחברות ונעילת חשבון · POST /api/auth/login

הקונטרולר מאמת את הגוף מול loginSchema ומעביר אימייל וסיסמה לשירות.

בשירות ארבעה כללים:

- המשתמש נשלף לפי אימייל באותיות קטנות. אימייל שאינו קיים וסיסמה שגויה מחזירים אותה שגיאה: INVALID_CREDENTIALS עם HTTP 401.
- אם lockUntil עדיין בעתיד, הבקשה נדחית לפני השוואת הסיסמה, עם HTTP 423 והקוד ACCOUNT_LOCKED.
- כל כישלון מגדיל את failedLoginAttempts. בכישלון החמישי (MAX_FAILED_LOGINS) נקבע lockUntil לעוד 15 דקות, נרשמת פעולת user.lockout, ומוחזר 423.
- התחברות מוצלחת מאפסת את המונה ואת הנעילה, רשמת user.login, ומנפיקה JWT.

apps/backend/src/auth/auth.service.ts

```

if (user.lockUntil && user.lockUntil.getTime() > Date.now()) {
  throw new HttpException(
    { code: ACCOUNT_LOCKED_CODE, message: 'Account locked after too many failed attempts' },
    HttpStatus.LOCKED,
  );
}
const match = await bcrypt.compare(password, user.passwordHash);
if (!match) {
  user.failedLoginAttempts += 1;
  if (user.failedLoginAttempts >= MAX_FAILED_LOGINS) {
    user.lockUntil = new Date(Date.now() + LOCK_MS);
  }
}

```

איור 3 — הנעילה ב-AuthService.login

הסיסמה נשמרת רק כ-hash של bcrypt בעשרה סבבי salt. ההשוואה היא bcrypt.compare מול ה-hash.

הרשמה של כתובת קיימת מחזירה 409 עם EMAIL_TAKEN. תשובת 423 על חשבון נעול מעידה שהחשבון קיים. זו הגנה חלקית ומודעת: הסתרה מלאה הייתה דורשת תשובה אחידה גם בהרשמה, במחיר בהירות למשתמש שנרשם.

5.3 אימות GET Token · api/auth/me

זו נקודת הקצה שה-useEffect של AuthProvider קורא לה בכל עליית אפליקציה, אם נשמר Token.

ה-JWT נחתם עם מזהה המשתמש, האימייל והתפקיד. תוקף ברירת המחדל הוא שמונה שעות (JWT_EXPIRES_IN).

5.4 שאר מנגנוני המודול

כל נתיב מוגן נבדק ב-JWT, והקונטרולר מקבל משתמש שכבר אומת. הסיסמה נשמרת רק כ-hash. שגיאות חוזרות כ-`{ success: false, error }`, והרשמה, התחברות, נעילה והתנתקות נרשמות ב-auditlogs.

5.5 החיבור לצד הלקוח

הדפים LoginPage ו-RegisterPage שולחים את הטופס לנקודות הקצה, ו-AuthProvider שומר את ה-Token ומעדכן את המשתמש. מסך ההתחברות מציג את הודעת הנעילה כשהשרת מחזיר 423. אחרי הצלחה המשתמש חוזר לכתובת שממנה הופנה, או לדף הבית.

פרק 6 — צד השרת: מודול הניהול

אין בשרת מודול בשם `admin` שמכיל את כל פעולות הניהול. ההגנה משותפת: `AuthGuard` ו-`AdminGuard`. הפעולות עצמן יושבות על מוצרים, על הזמנות ועל אנליטיקה. בצד הלקוח הן מתכנסות למסך אחד.

6.1 שלוש השכבות

רכיב ה-`AdminGuard` בודק `req.user.role === 'admin'` ומחזיר 403 עם `FORBIDDEN` לכל תפקיד אחר. הוא רץ אחרי שה-`Token` אומת, כי הוא תלוי ב-`payload` שכבר הוצמד לבקשה.

apps/backend/src/auth/admin.guard.ts

```
canActivate(context: ExecutionContext): boolean {
  const req = context.switchToHttp().getRequest<{ user?: JwtPayload }>();
  if (req.user?.role !== 'admin') {
    throw new ForbiddenException({ code: 'FORBIDDEN', message: 'Admin only' });
  }
  return true;
}
```

איור 4 — הבדיקה ב-`AdminGuard`

שכבה	קובץ	אחריות
Controller	products/ products.controller.ts	רשימת מנהל, יצירה, עדכון, מחיקה רכה
Controller	orders/ orders.controller.ts	רשימת הזמנות, שינוי סטטוס, ביטול
Controller	analytics/ analytics.controller.ts	GET /api/admin/stats
Service	products.service.ts	קטלוג, Soft Delete, התראת מלאי
Service	orders.service.ts	מכונת מצבים ו- <code>restock</code>
Service	analytics.service.ts	שש שאליות במקביל
Guard	auth/admin.guard.ts	דחיית משתמש שאינו מנהל
Audit	audit/ audit.service.ts	רישום פעולות יצירה, עדכון, מחיקה ושינוי סטטוס

טבלה 7 — חלוקת האחריות במודול הניהול

6.2 אישור וביטול הזמנה

המנהל רואה את ההזמנות ויכול לאשר אותן או לבטל אותן לפני המשלוח. אישור מעביר את ההזמנה ל-confirmed. ביטול מעביר אותה ל-cancelled ומחזיר את המלאי. אחרי שההזמנה יצאה למשלוח אי אפשר לבטל אותה.

6.3 שינוי סטטוס הזמנה · PATCH /api/orders/{id}/status

זו הפעולה הרגישה בלוח הניהול, כי היא משנה את סטטוס ההזמנה. הקונטרולר מוגן ב-AdminGuard, מחלץ את מזהה ההזמנה ואת גוף הבקשה, וקורא ל-OrdersService.updateStatus.

בשירות שתי בדיקות לפני כל כתיבה. הראשונה מוודאת שהסטטוס המבוקש הוא ערך מתוך ORDER_STATUSES. השנייה קוראת ל-canTransition. מעבר שאינו במטריצה נדחה עם ILLEGAL_TRANSITION. המעברים החוקיים:

אל	מ
cancelled, confirmed	pending_payment
cancelled, processing	confirmed
cancelled, shipped	processing
delivered	shipped
אין	delivered
אין	cancelled

טבלה 8 — מטריצת מעברי ההזמנה

מעבר ל-cancelled מחזיר את המלאי בטרנזקציה. אחרי משלוח אי אפשר לבטל, והשירות חוסם החזרת מלאי אם הסטטוס כבר delivered, shipped או cancelled.

6.4 שאר מנגנוני המודול

- בדיקת ההרשאה. הקישור לניהול מוצג רק למנהל. פנייה ישירה ל-/api/products/admin או ל-/api/orders/admin עם Token של לקוח נענית ב-403.
- Soft Delete. DELETE /api/products/{id} אינו מוחק את המסמך. הוא קובע isActive: false. הזמנות ישנות שומרות את שם המוצר ואת מחירו ב-snapshot, והמוצר נעלם מהקטלוג הציבורי.

- החזרה למלאי. עדכון שמעלה `stock` מאפס ליחידה חיובית שולח התראת דוא"ל לכל כתובת ב-`stockNotifyEmails` ומרוקן את הרשימה. בלי `MAIL_WEBHOOK_URL` ההודעה נכתבת ללוג השרת.

- **Audit Log**. יצירת מוצר, עדכון, מחיקה רכה ושינוי סטטוס נרשמים עם מזהה המנהל והמשאב.

- סינון הזמנות. `GET /api/orders/admin?status=` מחזיר את כל ההזמנות או רק סטטוס אחד.

6.5 החיבור לצד הלקוח

מסך ה-`AdminPage` מחזיק שלושה טאבים ב-`state` מקומי: `Products`, `Orders`, `Statistics`. כל טאב קורא לנקודת הקצה המתאימה. טופס הוספת המוצר רץ מול אותה סכמת `Zod` שבשרת, כדי להציג שגיאות שדה לפני השליחה. תרשים העמודות של `Recharts` מצייר את `salesByDay`.

פרק 7 — עגלה, הזמנה ומחזור חיים

7.1 עגלת קניות

העגלה נשמרת במסד הנתונים, לא בדפדפן. סגירת הדפדפן או מעבר מכשיר אינם מוחקים אותה, כל עוד המשתמש מחובר לאותו חשבון.

- שמירה כפולה. כל שינוי נכתב קודם ב-MongoDB, ואחר כך לעותק קצר בזיכרון השרת. המפתח הוא `cart: {userId}`, והעותק נשמר כחמש דקות. אם העותק לא זמין, הקריאה חוזרת למסד והעגלה לא הולכת לאיבוד.

- בדיקת מלאי בהוספה. המוצר חייב להיות פעיל ובעל `stock > 0`, והכמות אינה יכולה לעבור את המלאי. אחרת חוזר `OUT_OF_STOCK` או `NOT_FOUND`.

apps/backend/src/carts/carts.service.ts

```
const product = await this.products.findOne({
  _id: productId,
  isActive: true,
  stock: { $gt: 0 },
});
if (!product) {
  throw new NotFoundException({ code: 'NOT_FOUND', message: 'Product not available' });
}
if (quantity > product.stock) {
  throw new BadRequestException({ code: 'OUT_OF_STOCK', message: 'Not enough stock' });
}
```

איור 5 — בדיקת המלאי ב-`upsertItem`

- יחס 1:1. אינדקס ייחודי על `userId`.

- מחיר לתצוגה. בעת טעינת העגלה השירות מצרף שם, תמונה ומחיר עדכני מהקטלוג. נעילת המחיר מתרחשת רק ביצירת ההזמנה.

7.2 יצירת הזמנה

הקופה (`CheckoutPage`) עוברת משלושה שלבים בממשק אחד: סיכום העגלה, בחירה או יצירה של כתובת, ושליחת `POST /api/orders/checkout` עם `addressId` ו-`idempotencyKey`. אחרי הכתובת מופיע כפתור `Place order`. הסכום הוא סכום שורות ה-`snapshot`.

יצירת ההזמנה רצה בטרנזקציית MongoDB:

- אם אותו מפתח `idempotency` כבר מצביע על הזמנה של אותו משתמש, מוחזרת ההזמנה.

הקיימת ולא נוצרת שנייה.

- הכתובת חייבת להיות בבעלות המשתמש.

3. לכל שורה מתבצע `findOneAndUpdate` שמצליח רק אם המוצר פעיל והמלאי מספיק,

ומפחית את המלאי באותה פעולה.

4. נוצרת הזמנה בסטטוס `pending_payment` עם מספר `BLM-n`, שורות `snapshot` וסכום.

5. העגלה במסד מתרוקנת, ואז גם העותק הקצר בזיכרון.

apps/backend/src/orders/orders.service.ts

```
const product = await this.products
  .findOneAndUpdate(
    { _id: item.productId, isActive: true, stock: { $gte: item.quantity } },
    { $inc: { stock: -item.quantity } },
    { new: true, session },
  );
if (!product) {
  throw new BadRequestException({
    code: 'OUT_OF_STOCK',
    message: `Insufficient stock for ${item.name}`,
  });
}
snapshots.push({
  productId: product._id,
  name: product.name,
  imageUrl: product.imageUrl,
  unitPrice: product.price,
  quantity: item.quantity,
});
total += product.price * item.quantity;
```

איור 6 — הורדת מלאי ונעילת מחיר ב־`checkout`

כישלון באמצע הטרנזקציה מחזיר את המלאי ואת העגלה לאחור. לחיצה כפולה על יצירת ההזמנה נעצרת במפתח ה־`idempotency`, שנשמר 24 שעות. אישור ההזמנה הוא מעבר המנהל ל־`confirmed`.

7.3 מחזור חיים של הזמנה

סטטוס ההזמנה הוא `enum` בן שישה ערכים, והמעברים הם אלו שבטבלה 8. מספר ההזמנה מגיע ממונה אטומי בקולקציית `sequences`.

libs/shared-types/src/index.ts

```
export const ORDER_TRANSITIONS: Record<OrderStatus, OrderStatus[]> = {
  pending_payment: ['confirmed', 'cancelled'],
  confirmed: ['processing', 'cancelled'],
  processing: ['shipped', 'cancelled'],
  shipped: ['delivered'],
  delivered: [],
  cancelled: [],
};

export function canTransition(from: OrderStatus, to: OrderStatus): boolean {
  return ORDER_TRANSITIONS[from].includes(to);
}
```

איור 7 — מטריצת המעברים ב־shared-types

פריטי ההזמנה נשמרים כ־snapshot: productId, name, imageUrl, unitPrice, quantity. שינוי מחיר בקטלוג אחרי הקופה אינו משנה הזמנה שכבר נוצרה. ביטול אפשרי מ־pending_payment, מ־confirmed ומ־processing. הלקוח קורא ל־POST /api/orders/:id/cancel. המנהל יכול גם לבטל דרך שינוי הסטטוס. המלאי כבר ירד בקופה, ולכן הביטול מחזיר לכל שורה את הכמות שנשמרה ב־snapshot, בתוך טרנזקציה. אם מוצר היה באפס מלאי לפני ההחזרה, נשלחת התראת חזרה למלאי.

apps/backend/src/orders/orders.service.ts

```
await session.withTransaction(async () => {
  for (const item of order.items) {
    await this.products.updateOne(
      { _id: item.productId },
      { $inc: { stock: item.quantity } },
      { session },
    );
  }
  await this.orders.updateOne(
    { _id: order._id },
    { $set: { status: 'cancelled' } },
    { session },
  );
});
```

איור 8 — החזרת המלאי ב־cancelAndRestock

מ־shipped המעבר היחיד הוא אל delivered. אין החזרת מלאי אחרי יציאה למשלוח.

פרק 8 — נקודות הקצה

8.1 נקודות הקצה

הבסיס המקומי הוא `http://localhost:3030/api`.

תיאור	נתיב	פעולה
יצירת לקוח	<code>/api/auth/register</code>	POST
התחברות	<code>/api/auth/login</code>	POST
התנתקות	<code>/api/auth/logout</code>	POST
המשתמש המחובר	<code>/api/auth/me</code>	GET
קטלוג	<code>/api/products</code>	GET
פרטי מוצר	<code>/api/products/:id</code>	GET
התראת מלאי	<code>/api/products/:id/stock-alerts</code>	POST
כל המוצרים	<code>/api/products/admin</code>	GET
יצירת מוצר	<code>/api/products</code>	POST
עדכון מוצר	<code>/api/products/:id</code>	PATCH
מחיקה רכה	<code>/api/products/:id</code>	DELETE
שליפת עגלה	<code>/api/cart</code>	GET
עדכון פריט	<code>/api/cart/items</code>	PUT
הסרת פריט	<code>/api/cart/items/:productId</code>	DELETE
רשימת כתובות	<code>/api/addresses</code>	GET
יצירת כתובת	<code>/api/addresses</code>	POST
יצירת הזמנה	<code>/api/orders/checkout</code>	POST
ההזמנות שלי	<code>/api/orders/mine</code>	GET
פרטי הזמנה	<code>/api/orders/:id</code>	GET
ביטול הזמנה	<code>/api/orders/:id/cancel</code>	POST
כל ההזמנות	<code>/api/orders/admin</code>	GET
שינוי סטטוס	<code>/api/orders/:id/status</code>	PATCH
לוח בקרה	<code>/api/admin/stats</code>	GET
בדיקת חיות	<code>/api/health</code>	GET
ספר הפרויקט	<code>/api/docs</code>	GET
מדריך ה-API	<code>/api/docs/readme</code>	GET

טבלה 9 — נקודות הקצה

פרק 9 — תהליך הפיתוח, בדיקות והפצה

9.1 תהליך הפיתוח

שלושת הפרויקטים חיים בריפו אחד. החוזה הוגדר קודם ב-`libs/shared-types`: סכמות `Zod`, סטטוסים, ומטריצת המעברים. השרת והלקוח מייבאים את אותם טיפוסים, ו-`Swagger` משקף את הנתיבים.

פקודות העבודה, תמיד דרך `Nx` המקומי:

• `npm run dev` — שרת על פורט 3030 וחנות על פורט 3000.

• `npx nx run-many -t lint,build,test,typecheck` של `frontend`, `backend` ו-`shared-types`.

בהפעלה אפשר להוסיף חשבון מנהל אחד ושני חשבונות לקוח. הם נוצרים כשמוגדרים גם האימייל וגם הסיסמה.

תפקיד	אימייל	סיסמה
מנהל	<code>avivedri3@gmail.com</code>	<code>AvivAdmin2026!</code>
לקוח	<code>demo.maya@bloomstore.com</code>	<code>DemoMaya2026!</code>
לקוח	<code>demo.alex@bloomstore.com</code>	<code>DemoAlex2026!</code>

טבלה 10 — חשבון מנהל ושני חשבונות לקוח

9.2 מערך הבדיקות

בדיקות היחידה שרצות ב-`CI` מכסות את מכונת המצבים (`libs/shared-types`), את מעטפת ה-`HTTP` ואת מדיניות הנעילה (`apps/backend/src/common/http.spec.ts`), ואת רכיבי הממשק `Header`, `ProductCard`, `CategoryPills` ו-`CatalogHero`. תרחישי הקבלה שלהלן הם מערך הבדיקה הידני של המערכת. העמודה האחרונה מציינת איפה הכלל מעוגן בקוד, ולא תוצאה של הרצה שנרשמה מחוץ ל-`CI`.

#	תרחיש	צעדים	תוצאה מצופה	כיסוי
1	הרשמה תקינה	שם, אימייל וסיסמה באורך 8 לפחות	נוצר לקוח, מונפק <code>Token</code> , המשתמש מחובר	<code>AuthService.register</code>
2	נעילת חשבון	חמישה ניסיונות עם סיסמה שגויה	HTTP 423, נעילה ל-15 דקות	<code>MAX_FAILED_LOGIN</code> , בדיקת יחידה למדיניות

#	תרחיש	צעדים	תוצאה מצופה	כיסוי
3	מוצר לא פעיל	<code>isActive: false</code> ורענון הקטלוג	המוצר אינו מוצג	<code>listPublic</code>
4	מוצר שאזל	<code>stock === 0</code>	המוצר מוצג, אי אפשר להוסיף לעגלה, אפשר להירשם להתראה	<code>,ProductCard</code> <code>CartsService</code>
5	התמדת העגלה	הוספה, סגירת דפדפן, התחברות מחדש	הפריטים נטענים מ-MongoDB	<code>CartProvider</code>
6	חריגה מהמלאי	כמות גדולה מהמלאי	400 <code>OUT_OF_STOCK</code>	<code>CartsService.upsertItem</code>
7	נעילת מחיר	שינוי מחיר בקטלוג אחרי קופה	סכום ההזמנה נשאר המחר שב-snapshot	<code>OrdersService.checkout</code>
8	הזמנה כפולה	אותו <code>idempotencyKey</code> פעמיים	הזמנה אחת	<code>idempotencykeys</code>
9	גישת לקוח לניהול	<code>GET /api/products/</code> עם <code>admin Token</code> של לקוח	403	<code>AdminGuard</code>
10	מעבר סטטוס פסול	מ- <code>shipped</code> אל <code>cancelled</code>	<code>ILLEGAL_TRANSITION</code>	<code>canTransition</code> , בדיקת יחידה
11	ביטול לפני משלוח	ביטול מ- <code>confirmed</code>	המלאי חוזר	<code>cancelAndRestock</code>
12	מפתח הזרקה	שדה שמתחיל ב-\$ בגוף הבקשה	המפתח מוסר לפני השירות	<code>MongoSanitizeMiddleware</code>

טבלה 11 — מערך הבדיקות

זרימת CI (`.github/workflows/ci.yml`) רצה על `main`, על `develop` ועל `pull request` אל `main`: `checkout`, התקנת תלויות, `lint`, `typecheck`, `test`, `build` ו-`npm audit`, על הפרויקטים המושפעים ב-Nx.

9.3 תהליך ההפצה

שני יעדים נפרדים. החנות וספר הפרויקט הם אתר סטטי. ה-API הוא שירות Docker.

GitHub Pages מגיש את קבצי ה-React בלי שרת משלה. האתר יושב ב-`https://avivedri3.github.io/bloomstore/`, ולידו גם עמוד הספר וקובץ ה-Word. החנות קוראת ל-API שב-Render לפי הכתובת שנקבעה ב-`VITE_API_BASE_URL`.

מה הוא עושה	מנגנון
איכות הקוד לפני ואחרי מיזוג	<code>ci.yml</code>
בונה את ה-SPA, ממיר את הספר ל-HTML ובשלב נוסף ל-DOCX, ומפרסם את שניהם ל-GitHub Pages אחרי שה-workflow של השרת הצליח	<code>deploy-gh-pages.yml</code>
בונה את <code>apps/backend/Dockerfile</code> מהשורש ומפרסם בכל דחיפה ל- <code>main</code> (<code>autoDeployTrigger: commit</code>)	Render Blueprint (<code>render.yml</code>)
מאמת <code>nx build</code> של השרת. אם הוגדר הסוד <code>RENDER_DEPLOY_HOOK</code> , הוא גם קורא ל- <code>hook</code> . כש- <code>auto-deploy</code> של Render דולק, משאירים את ה- <code>hook</code> ריק כדי לא לבנות פעמיים	<code>deploy-backend.yml</code>

טבלה 12 — תהליך ההפצה

כתובות:

• חנות: `https://avivedri3.github.io/bloomstore/`

• ספר Word: `https://avivedri3.github.io/bloomstore/docs/project-book.docx`

• API: שירות Render בשם `bloomstore`, אזור `Frankfurt`, תוכנית חינמית. בדיקת החיות היא `GET /api/health`

• אחרי העלייה מגדירים את משתנה ה-`VITE_API_BASE_URL` Actions לכתובת של שירות Render, עם הסיימת `/api`, ומריצים שוב את `Deploy to GitHub Pages`.

משתני סביבה קריטיים ב-Render: `MONGODB_URI` ו-`JWT_SECRET` (לפחות 32 תווים). מומלצים גם `CORS_ORIGINS=https://`, `JWT_EXPIRES_IN=8h`, `DB_NAME=bloomstore`, `FRONTEND_URL=https://avivedri3.github.io/`, `avivedri3.github.io/bloomstore/`. בלי `MONGODB_URI` או `JWT_SECRET` התהליך נעצר בעלייה. טרנזקציות הביטול דורשות `replica set`, ו-Atlas מספק אותו. ב-`Network Access` של Atlas יש לאפשר את `Render` כתובות.

פרק 10 — סיכום ומסקנות

10.1 הישגים טכניים

- חנות Full-Stack מלאה במונורפו Nx: קטלוג, חשבון, עגלה, קופה, הזמנות ולוח ניהול.
- מודול הזדהות עם נעילת חשבון אחרי חמישה כישלונות.
- ניהול מוגן ב-Guard, עם Soft Delete, יומן ביקורת ודשבורד שמריץ שאילתות במקביל.
- עגלה שנשמרת ב-MongoDB ונשארת גם אחרי סגירת הדפדפן.
- נעילת מחיר בשורת ההזמנה, מכונת מצבים, והחזרת מלאי בטרנזקציה לפני משלוח.
- מראה רגוע לחנות: קרם, ורד מאובק ומרווה, עם DM Sans ו-Fraunces.
- הפצה כפולה: GitHub Pages לחנות ולספר, Render ל-API.
- API מתועד ב-Swagger, וספר הפרויקט נגיש ב-GET /api/docs.

10.2 אתגרים ומה שנלמד

בשרת היה צריך לבחור כמה זמן Access Token נשאר בתוקף, בלי מנגנון רענון. שמונה שעות הן פשרה. תוקף קצר מנתק את הלקוח באמצע קנייה, ותוקף ארוך משאיר את החיבור פתוח זמן רב. נעילה אחרי חמישה כישלונות שומרת על החשבון כשמישהו מנסה סיסמאות שוב ושוב.

בצד הלקוח האתגר היה שהעגלה והמשתמש נקראים מדפים רחוקים. העברת הנתונים דרך props בכל רמה שוברת את העקביות ברגע שמוסיפים מסך. ריכוזם ב-Context, מעל App, נתן לכל דף את אותו מצב.

מהעבודה נלמד שהכלל העסקי צריך לשבת במקום אחד. צילום המחיר בשורת ההזמנה מונע פער בין מה שהלקוח אישר לבין מחיר שהשתנה אחר כך בקטלוג. מכונת המצבים חוסמת מעבר אסור, למשל ביטול אחרי משלוח. ביטול בלי טרנזקציה היה משנה סטטוס בלי להחזיר מלאי. הסדר Controller, אחר כך Service, ואחר כך Model הוא חלק מהנכונות: הקונטרולר לא נוגע במסד, והשירות לא בונה את תשובת ה-HTTP.

עוד לקח עלה בהפצה. החנות וה-API רצים בנפרד, ב-GitHub Pages וב-Render, והחווה ב-libs/shared-types הוא מה שמחזיק אותם יחד. מוצר שאזל נשאר בקטלוג כדי שאפשר להירשם להתראה, והעגלה עדיין דוחה אותו. ההבחנה נראית קטנה בממשק, והיא אותו כלל גם בשרת.

10.3 פיתוחים עתידיים

1. ה-Token תקף שמונה שעות, בלי מנגנון חידוש. קיצור התוקף, יחד עם Refresh בצד הלקוח, ישאיר את הלקוח מחובר לאורך הקנייה בלי להשאיר את החיבור פתוח זמן רב.
2. הבדיקות ב-CI הן בדיקות יחידה. חבילת E2E שרצה על תרחישי טבלה 11 בכל דחיפה תעבור את המסלול מהקטלוג עד ההזמנה, ולא רק פונקציה אחת.
3. הקטלוג מסונן היום לפי קטגוריה. חיפוש טקסט מלא יאפשר למצוא זר לפי שם או לפי תיאור.
4. דירוג קצר וביקורת יוכלו להופיע ליד כרטיס המוצר. המחיר שננעל בשורת ההזמנה יישאר כפי שנשמר ברגע ההזמנה.
5. MailService שולח היום התראה כשמוצר חוזר למלאי. אותו מנגנון יכול לשלוח דוא"ל גם כשסטטוס ההזמנה משתנה, למשל כשהיא יוצאת למשלוח.

10.4 תודות

תודה למור ברגיג על הליווי לאורך הפרויקט. למכללת אורט סינגאלובסקי תודה על הכלים ועל הידע שנרכשו במהלך הלימודים, ולקהילת הקוד הפתוח על הספריות ועל התייעוד שעליהם עומדת BloomStore.

מקורות

- עיצוב הממשק בספריית MUI, לפי `apps/frontend/src/theme/tokens.ts`
- `render.yaml` וקבצי ה-workflow תחת `..github/workflows`